



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К КУРСОВОЙ РАБОТЕ

НА ТЕМУ:

«Мониторинг аппаратных ресурсов»

Студент ИУ7-74Б
(Группа)

(Подпись, дата)

Сальников М. А.
(И. О. Фамилия)

Руководитель курсовой работы

(Подпись, дата)

Рязанова Н. Ю.
(И. О. Фамилия)

2025 г.

СОДЕРЖАНИЕ

РЕФЕРАТ	5
ОПРЕДЕЛЕНИЯ	6
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ	7
ВВЕДЕНИЕ	8
1 Аналитический раздел	9
1.1 Формализация задачи	9
1.2 Загружаемые модули ядра Linux	9
1.3 Известные характеристики использования процессора	10
1.3.1 Процессорное время	10
1.3.2 Тактовая частота	11
1.3.3 Количество инструкций за такт	12
1.3.4 Прерывания	13
1.3.5 Аппаратный контекст	13
1.4 Известные характеристики использования оперативной памяти	13
1.5 Анализ известных способов сбора данных об использовании процессора	13
1.5.1 /proc	13
1.5.2 /sys	13
1.5.3 struct kernel_cpustat	13
1.5.4 cpufreq	13
1.5.5 perf_event	13
1.6 Анализ известных способов сбора данных об использовании оперативной памяти	13
1.7 Виртуальная файловая система /proc	13
2 Конструкторский раздел	14
3 Технологический раздел	15
ЗАКЛЮЧЕНИЕ	16

ПРИЛОЖЕНИЕ А	17
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	17

РЕФЕРАТ

Расчетно-пояснительная записка № с., № рис., № лист., № ист., № прил.

ОПРЕДЕЛЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие термины с соответствующими определениями.

Термин — определение

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

В настоящей расчетно-пояснительной записке применяют следующие сокращения и обозначения.

CPU — Central Processing Unit

API — Application Programming Interface

IPC — Instructions Per Cycle

ВВЕДЕНИЕ

Целью данной курсовой работы является разработка загружаемого модуля ядра Linux, предоставляющего пользователю процессу информацию об использовании системой процессора и оперативной памяти.

Для достижения данной цели необходимо выполнить следующие задачи:

- проанализировать известные характеристики использования процессора;
- проанализировать известные характеристики использования оперативной памяти;
- проанализировать известные способы сбора данных об использовании процессора;
- проанализировать известные способы сбора данных об использовании оперативной памяти;
- спроектировать алгоритмы для достижения поставленной цели;
- спроектировать структуру загружаемого модуля;
- выбрать наиболее подходящий способ взаимодействия с загружаемым модулем;
- выбрать средства реализации загружаемого модуля и реализовать его.

1 Аналитический раздел

В данном разделе представляются формализация задачи, описания загружаемых модулей Linux, известных характеристик использования процессора и оперативной памяти и анализ известных способов сбора данных об использовании системой процессора и оперативной памяти.

1.1 Формализация задачи

Необходимо реализовать загружаемый модуль ядра Linux, предоставляющий информацию из пространства ядра в пространство пользователя об использовании системой за определенный промежуток времени процессора и оперативной памяти.

1.2 Загружаемые модули ядра Linux

Загружаемый модуль ядра представляет собой компилированный объектный код, предназначенный для динамического расширения функциональности ядра операционной системы без необходимости его перекомпиляции. Данный механизм позволяет инкапсулировать новый код в независимые компоненты, которые могут быть присоединены к исполняемому образу ядра во время его работы или отсоединены от него.

Процесс интеграции модуля заключается в вызове его функции инициализации, которая регистрирует предоставляемые модулем услуги в соответствующих подсистемах ядра. После успешной регистрации функция инициализации завершает работу, а модуль становится частью исполняемого кода ядра. Корректное удаление модуля инициирует выполнение функции завершения, задача которой – осуществить deregистрацию услуг и освободить все занятые системные ресурсы для обеспечения целостности ядра.

Важным аспектом архитектуры модулей является их зависимость от интерфейсов, экспортируемых самим ядром: модуль не может использовать функции, не объявленные в его публичном API. Это ограничение обеспечивает стабильность системы, но создает зависимость модуля от конкретной версии ядра. Основное преимущество использования загружаемых модулей заключается в значительном сокращении цикла разработки и отладки низкоуровневого кода, так как процесс итеративного тестирования не требует

перезапуска всей системы.

1.3 Известные характеристики использования процессора

Операционная система использует центральный процессор в качестве ключевого вычислительного ресурса для координации работы всех компонентов системы. Для оценки использования процессора может быть применен набор количественных и качественных характеристик.

1.3.1 Процессорное время

Процессорное время характеризует интервал времени, в течение которого процессор выполнял конкретную задачу. Операционная система классифицирует процессорное время, учет которого ведется для каждого процессорного ядра в отдельности. Классы процессорного времени представлены в таблице 1.1.

Таблица 1.1 – Классы процессорного времени

Класс	Описание
Пользовательское время	Время, затраченное на исполнение кода программ в пространстве пользователя
Системное время	Время, затраченное на исполнение кода операционной системы в режиме ядра
Время ожидания ввода-вывода	Время, в течение которого процессор простаивал в состоянии готовности, ожидая завершения операций ввода-вывода от устройств
Время обработки прерываний	Время, затраченное на обслуживание аппаратных прерываний
Время обработки отложенных прерываний	Время, затраченное на обработку нижних половин прерываний
Время ожидания	Время, в течение которого процессорное ядро было полностью свободно

Для оценки относительной загрузки процессора U_{CPU} за выбранный интервал используется показатель, вычисляемый как отношение времени полезной работы к полному времени:

$$U_{CPU} = \left(1 - \frac{I_{end} - I_{begin}}{T_{end} - T_{begin}}\right) \times 100\% \quad (1.1)$$

где:

1. I_{end} и I_{begin} – время ожидания в конце и начале интервала соответственно;
2. T_{end} и T_{begin} – полное время в конце и начале интервала соответственно.

1.3.2 Тактовая частота

Такт ядра процессора – промежуток между двумя импульсами тактового генератора, который синхронизирует выполнение всех операций процессора. Выполнение различных элементарных операций может занимать от долей такта до многих тактов в зависимости от команды и процессора. Тактовая частота является физической характеристикой процессора, определяющей количество синхронизирующих тактов, выполняемых за одну секунду.

Современным процессорам свойственна способность к динамическому изменению тактовой частоты во время работы. Данный механизм служит для решения двух основных задач: повышения энергоэффективности и управления тепловыделением. Снижение частоты в периоды низкой вычислительной нагрузки позволяет значительно уменьшить энергопотребление и нагрев кристалла, а кратковременное повышение частоты сверх номинала позволяет увеличить производительность, если это позволяет тепловой пакет и система питания.

Ответственность за динамическое изменение тактовой частоты распределена между аппаратным обеспечением и программным обеспечением. Непосредственно генерация тактовой частоты осуществляется аппаратным блоком – тактовым генератором, а логика принятия решения о необходимости и величине изменения частоты может реализовываться как аппаратным обеспечением, так и операционной системой.

1.3.3 Количество инструкций за такт

Количество инструкций за такт (IPC) характеризует эффективность использования процессором его вычислительных ресурсов в течение одного такта и может быть вычислена как:

$$IPC = \frac{N_{instructions}}{N_{cycles}} \quad (1.2)$$

где:

1. $N_{instructions}$ – общее количество исполненных процессором инструкций;
2. N_{cycles} – общее количество затраченных процессорных тактов.

Величина IPC не является постоянной, а изменяется динамически в зависимости от множества факторов. Наиболее значимое влияние оказывает микроархитектура процессора: количество и тип исполнительных устройств, размер и ассоциативность кэш-памяти всех уровней, эффективность предвыборки данных и команд. Со стороны программного обеспечения IPC определяется характером вычислительной нагрузки: интенсивностью и локальностью обращений к памяти, соотношением арифметических, целочисленных и операций с плавающей точкой, плотностью ветвлений.

Непосредственно изменить IPC ни операционная система, ни пользовательское приложение не могут, поскольку эта величина является производной от физических возможностей ядра и скомпилированного машинного кода. Однако разработчики могут косвенно влиять на него через оптимизацию программ: улучшение локальности данных, уменьшение количества условных переходов и выравнивание кода, что позволяет более полно загрузить вычислительные ресурсы процессора и, как следствие, повысить IPC.

1.3.4 Прерывания

1.3.5 Аппаратный контекст

1.4 Известные характеристики использования оперативной памяти

1.5 Анализ известных способов сбора данных об использовании процессора

1.5.1 /proc

1.5.2 /sys

1.5.3 struct kernel_cpustat

1.5.4 cpufreq

1.5.5 perf_event

1.6 Анализ известных способов сбора данных об использовании оперативной памяти

1.7 Виртуальная файловая система /proc

2 Конструкторский раздел

3 Технологический раздел

ЗАКЛЮЧЕНИЕ

ПРИЛОЖЕНИЕ А

Презентация к курсовой работе, содержащая n слайдов